HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

PROJECT REPORT

Flight Data Monitoring

Course: IT4043E - Big Data Storage and Processing

Supervisor: Dr. Tran Viet Trung

Authors:	Student ID:
Nguyen Duc Binh	20225475
Tran An Khanh	20225447
Pham Minh Hieu	20220062
Truong Tuan Vinh	20225464
Nguyen Nhu Giap	20225441

Hanoi, January 2026

ABSTRACT

This project presents a comprehensive real-time flight data streaming and analytics system leveraging modern big data technologies. The system implements a Kappa architecture to continuously process live flight telemetry from the OpenSky Network API, demonstrating the complete lifecycle of big data processing from ingestion to visualization. The architecture integrates Apache NiFi for data ingestion, Apache Kafka for stream transport, Apache Spark Structured Streaming for real-time processing, Apache Cassandra and MinIO for dual-path storage, and Apache Superset with Trino for analytics and visualization. The system exhibits all five characteristics of big data—volume, velocity, variety, veracity, and value—by processing tens of thousands of flights generating millions of events daily with updates every 1-5 seconds. The implementation provides operational insights including regional flight density analysis, real-time trajectory tracking, and time-series aggregations. Deployment capabilities span both local Docker Compose environments and distributed Kubernetes clusters, offering scalability and fault tolerance. This work demonstrates practical implementation of stream-oriented big data processing and serves as a reference architecture for real-time telemetry analytics systems.

Keywords: real-time streaming, Kappa architecture, Apache Kafka, Apache Spark, Cassandra, flight monitoring, big data analytics, distributed systems.

Contents

1 Introduction **4**

1.1 Project Background 4

1.2 Problem Statement 4

1.3 Project Objectives & Overall Architecture 5

2 Data Ecosystem and Characterization **6**

2.1 OpenSky Network (Primary Data Source) 6

2.2 Aviation Weather (Enrichment Source) 7

2.3 Static Reference Data 8

3 Methodology **8**

3.1 Data Ingestion Layer (Apache NiFi) 8

3.1.1 NiFi Flow Design 8

3.1.2 Flow Deployment Automation 12

3.2 Message Streaming Layer (Kafka & ZooKeeper) 13

3.2.1 Kafka Cluster Configuration 13

3.2.2 Kafka Topic and Client Configuration 14

3.3 Stream Processing Layer (Spark Structured Streaming) 14

3.3.1 Spark Application Design 14

3.3.2 Data Cleaning and Enrichment 16

3.3.3 Windowing, Watermarking, and Aggregations 16

3.4 Storage Layer (Dual Storage Strategy) 17

3.4.1 Cassandra (Operational / Hot path) 17

3.4.2 MinIO (Archival / Cold Path) 21

3.5 Analytics & Visualization Layer 22

3.5.1 Trino Query Engine 23

3.5.2 Apache Superset Visualization 24

3.6 Deployment & Operations Approach 27

4 Conclusion **30**

4.1 Summary of Achievements and Discussion 30

4.2 Future Work 31

References **33**

1 Introduction

1.1 Project Background

The global aviation industry produces a continuous stream of high-fidelity telemetry data from thousands of aircraft operating simultaneously. With the shift from traditional radar to satellite-based Automatic Dependent Surveillance-Broadcast (ADS-B) technology, flight data has become more accessible and detailed than ever. Community-driven initiatives like the OpenSky Network now provide open-access datasets that capture real-time global air traffic, offering a wealth of information for monitoring and analysis.

However, the raw data generated by these networks is characterized by high velocity, massive volume, and inherent noise. Traditional monolithic database systems often struggle to process and store such high-frequency updates in real-time. This creates a technical gap between raw data acquisition and meaningful situational awareness.

The **Flight Data Monitoring System** is developed to bridge this gap. By utilizing a distributed Big Data stack and the **Kappa Architecture**, the project aims to build a scalable pipeline that ingests raw ADS-B telemetry and transforms it into actionable insights. This system provides a robust framework for real-time air traffic monitoring, density analysis, and historical trend evaluation.

1.2 Problem Statement

The core challenge lies in designing a distributed data pipeline capable of ingesting a continuous stream of aircraft telemetry (including identifiers, coordinates, altitude, speed, and timestamps). This problem perfectly exemplifies the **5Vs of Big Data**:

- **Volume:** Processing millions of events daily generated by tens of thousands of concurrent flights.
- **Velocity:** Handling high update frequencies, requiring low-latency stream processing.
- **Variety:** Managing complex, semi-structured JSON data with nested arrays and heterogeneous fields.
- **Veracity:** Addressing data noise, signal interference, or missing coordinates through robust cleaning and validation mechanisms.
- **Value:** Providing critical operational answers regarding peak traffic density, average altitudes by airline/country, and real-time trajectory variations.

1.3 Project Objectives & Overall Architecture

The primary objective of this project is to design and deploy a scalable data pipeline based on the **Kappa Architecture**. The overall architecture is illustrated in Figure 1.

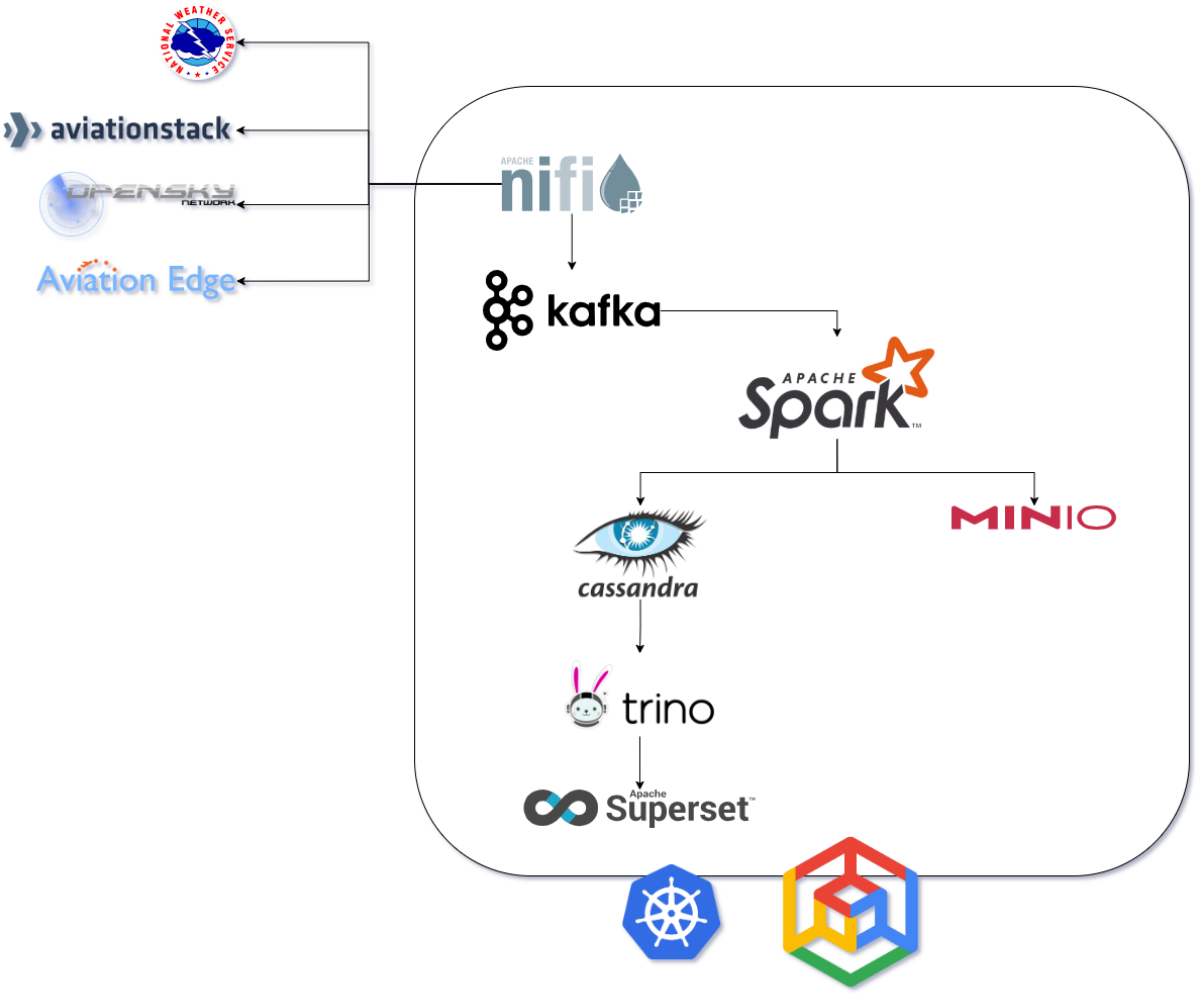

Figure 1: Overall architecture of our Flight Data Monitoring system.

Specific goals include:

1. **Automated Ingestion:** Implementing an automated pipeline to extract data from OpenSky Network APIs (States, Flights, and Tracks) and Aviation Weather Center APIs (METAR, TAF, SIGMET) using **Apache NiFi**.

2. **Distributed Message Streaming with Apache Kafka:** Deploying a fault-tolerant **Apache Kafka** cluster with Kafka brokers to provide reliable, high-throughput message brokering between ingestion and processing layers.

3. **Real-time Stream Processing:** Utilizing **Apache Spark Structured Streaming** to consume from Kafka topics for data cleaning, enrichment, and time-based windowing aggregations.

4. **Dual-Storage Strategy:**
    - Hot Storage: Storing operational data in **Apache Cassandra** for low-latency queries and real-time monitoring.
    - Cold Storage: Archiving historical data in **MinIO** (S3-compatible) using compressed **Parquet** format for long-term analytics.
5. **Analytics and Visualization:** Enabling SQL-based querying via **Trino** and deploying interactive dashboards on **Apache Superset** to visualize live maps and density heatmaps.
6. **Scalable Orchestration with Kubernetes:** Leveraging **Kubernetes** (K8s) and Cloud technologies to orchestrate critical stateful and stateless components.

2 Data Ecosystem and Characterization

The Flight Data Monitoring System integrates multiple aviation-related datasets to support near real-time monitoring and downstream analytics. This data ecosystem is strategically designed to cover live aircraft telemetry, historical flight context, and auxiliary datasets for enrichment and interpretability.

2.1 OpenSky Network (Primary Data Source)

The **OpenSky Network** serves as the primary high-throughput ingestion source for the platform. It provides a massive, open-access dataset capturing the pulse of global aviation through three specialized API endpoints, each contributing to the system's complexity and data volume.

Real-Time Aircraft States (`/api/states/all`) This endpoint constitutes the most intensive data stream, operating at a high velocity with update frequencies ranging from 5 to 10 seconds per aircraft. The data is delivered as a complex JSON array, capturing the instantaneous kinematics of tens of thousands of concurrent flights globally, which demands a highly scalable processing layer. The data features are included in Table 1.

Attribute	Type	Description
`icao24`	String	Unique 24-bit transponder address.
`callsign`	String	Flight identifier (cleaned for whitespace).
`origin_country`	String	Country of aircraft registration.
`longitude`	Float	WGS-84 decimal degree coordinates.
`latitude`	Float	WGS-84 decimal degree coordinates.
`baro_altitude`	Float	Barometric altitude (meters).
`velocity`	Float	Ground speed (meters/second).
`true_track`	Float	Heading (degrees clockwise from true north).
`vertical_rate`	Float	Rate of climb or descent (m/s).
`on_ground`	Boolean	True if the aircraft is at the airport surface.
`ingestion_ts`	String	Collection timestamp (added via Apache NiFi).

Table 1: State Vector Attributes

Historical Flights and Trajectories To complement real-time states, the system ingests flight and trajectory context:

- **Historical Flights (`/api/flights/all`):** To provide a broader context of aviation operations, this source is polled to capture completed flight segments within specific time intervals. This stream is characterized by its metadata richness, including estimated departure and arrival ICAO airport codes.
- **Trajectories / Tracks (`/api/tracks/all`):** This endpoint provides a deep dive into the 4D movement patterns of individual aircraft. Unlike the flat structure of state vectors, flight tracks return a nested array structure (path) representing a sequence of waypoints. Each waypoint includes a multidimensional set of attributes (time, latitude, longitude, barometric altitude, and ground status).

2.2 Aviation Weather (Enrichment Source)

To contextualize flight operations with meteorological conditions, the system integrates high-fidelity products from the **Aviation Weather Center (AWC) Data API**. This dataset serves as a critical enrichment layer, allowing the platform to correlate aircraft telemetry with real-world atmospheric hazards and terminal conditions.

The ingestion layer is configured to handle three primary data products, each routed to dedicated Kafka topics to ensure isolated and efficient processing:

- **METAR:** Actual meteorological observations at terminal stations, streamed into the `current_airport_weather` topic.
- **TAF:** Terminal Aerodrome Forecasts for predicted conditions, managed within the `future_airport_weather` topic.

- **SIGMET:** Significant Meteorological advisories regarding hazardous phenomena (e.g., turbulence, icing), routed to the `weather_warnings` topic.

2.3 Static Reference Data

To improve dashboard interpretability and normalize raw identifiers, the platform integrates static reference datasets sourced from the **Aviationstack API**.

- **Airlines Reference:** A comprehensive database of global airlines containing fleet information, operational metadata, and organizational identifiers. Each record includes IATA codes (e.g., AA, DL, UA), ICAO codes (e.g., AAL, DAL, UAL), radio callsigns, hub airport codes, country of registration, fleet size, average aircraft age, and operational status. This enables transformation of flight callsigns into airline names and association of aircraft for BI reporting and fleet analysis.

- **Airports Reference:** A global airport database providing geographic coordinates, timezone information, and multi-format identifiers (IATA, ICAO, GeoNames ID). Each airport record contains precise latitude/longitude coordinates in decimal degrees, IANA timezone identifiers, GMT offsets, country ISO codes, and human-readable location names. This dataset supports departure/arrival airport enrichment from ICAO codes (e.g., KJFK, EGLL) in flight connection records.

- **Cities Reference:** A supplementary dataset mapping IATA city codes to geographic coordinates and timezone information. This enables city-level aggregations, supports regional density statistics.

3 Methodology

3.1 Data Ingestion Layer (Apache NiFi)

Apache NiFi orchestrates data ingestion through visual dataflow programming, providing reliability, backpressure management, and operational monitoring capabilities.

3.1.1 NiFi Flow Design

Two parallel flow groups handle distinct data source APIs, each optimized for source characteristics, authentication requirements, and update frequencies.

OpenSky Network API Flow The OpenSky flow ingests real-time aircraft surveillance data through three parallel data streams: state vectors, flight connections, and trajectory tracks. The flow is illustrated in Figure 2. All streams share a common authentication infrastructure based on OAuth2 client credentials flow.

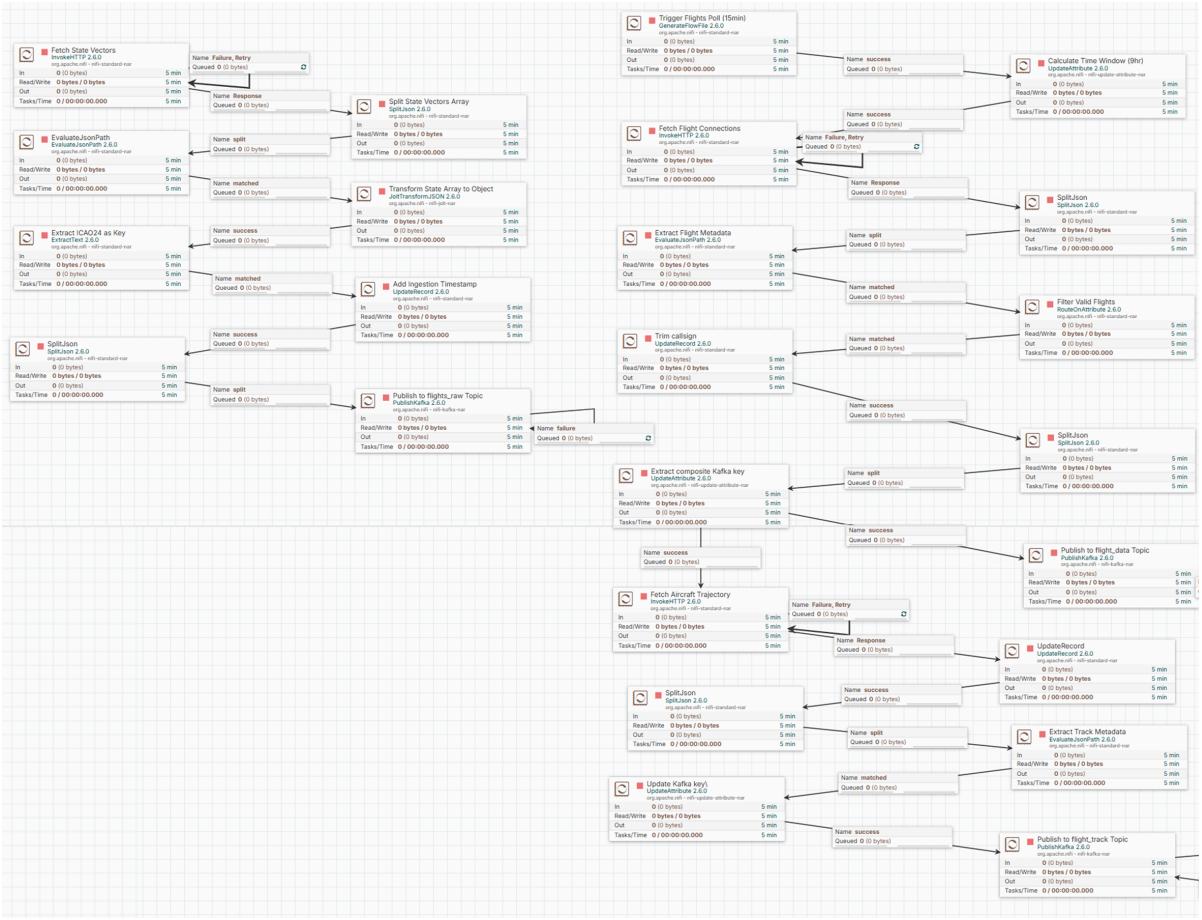

Figure 2: OpenSkyAPI flow in Nifi UI

Authentication Infrastructure: The `StandardOauth2AccessTokenProvider` controller service manages the OAuth2 lifecycle for elevated rate limits (4,000 API credits/-day vs. 400 for anonymous users) and access to historical data. The service authenticates against OpenSky Network using client credentials stored in environment variables, with automatic token renewal occurring 5 minutes before the 30-minute expiration window. Bearer tokens are transparently attached to all `InvokeHTTP` processor requests.

States Stream (`flights_raw` topic): A timer-driven `InvokeHTTP` processor polls `/api/states/all` endpoint every 20 seconds, capturing global aircraft position snapshots. The raw JSON response contains a two-dimensional array where each row represents a state vector with 18 fields (icao24, callsign, longitude, latitude, altitude, velocity, etc.). The `SplitJson` processor extracts individual state vectors, producing one FlowFile per aircraft. `JoltTransformJSON` applies shift transformations to map array indices to named fields, converting the compact array format to structured JSON objects. `UpdateRecord` processors trim whitespace from callsign fields and inject ingestion timestamps using NiFi Expression Language. `ExtractText` extracts ICAO24 transponder addresses as FlowFile attributes, which serve as Kafka message keys ensuring aircraft affinity to specific partitions. Finally, `PublishKafka` transmits messages to the `flights_raw` topic.

Flights Stream (`flight_data` topic): Operating at 15-minute intervals, this flow retrieves flight connection records via `/api/flights/all` endpoint. The `UpdateAttribute` processor dynamically computes query parameters using NiFi Expression Language, generates Unix timestamps for 9-hour lookback windows (32,400 seconds). The response contains flight metadata including estimated departure/arrival airports, first/last seen timestamps, and distance metrics. `RouteOnAttribute` filters records with non-null `estDepartureAirport` and `estArrivalAirport` fields, reducing downstream processing of incomplete flight connections. Valid records use composite keys (`icao24-firstSeen`) to enable time-based partitioning in Kafka.

Tracks Stream (`flight_track` topic): This flow fetches complete aircraft trajectories from `/api/tracks/all` endpoint using icao24 and time parameters. The response contains a waypoint array with position, altitude, and track information at variable time intervals. `EvaluateJsonPath` extracts the `startTime` field for composite key construction (`icao24-startTime`), partitioning trajectories by aircraft and flight session.

Aviation Weather API Flow The Aviation Weather flow ingests meteorological data from the FAA's Aviation Weather Center, providing METAR surface observations, TAF terminal forecasts, and SIGMET hazard warnings for correlation with flight operations. The full flow is illustrated in Figure 3.

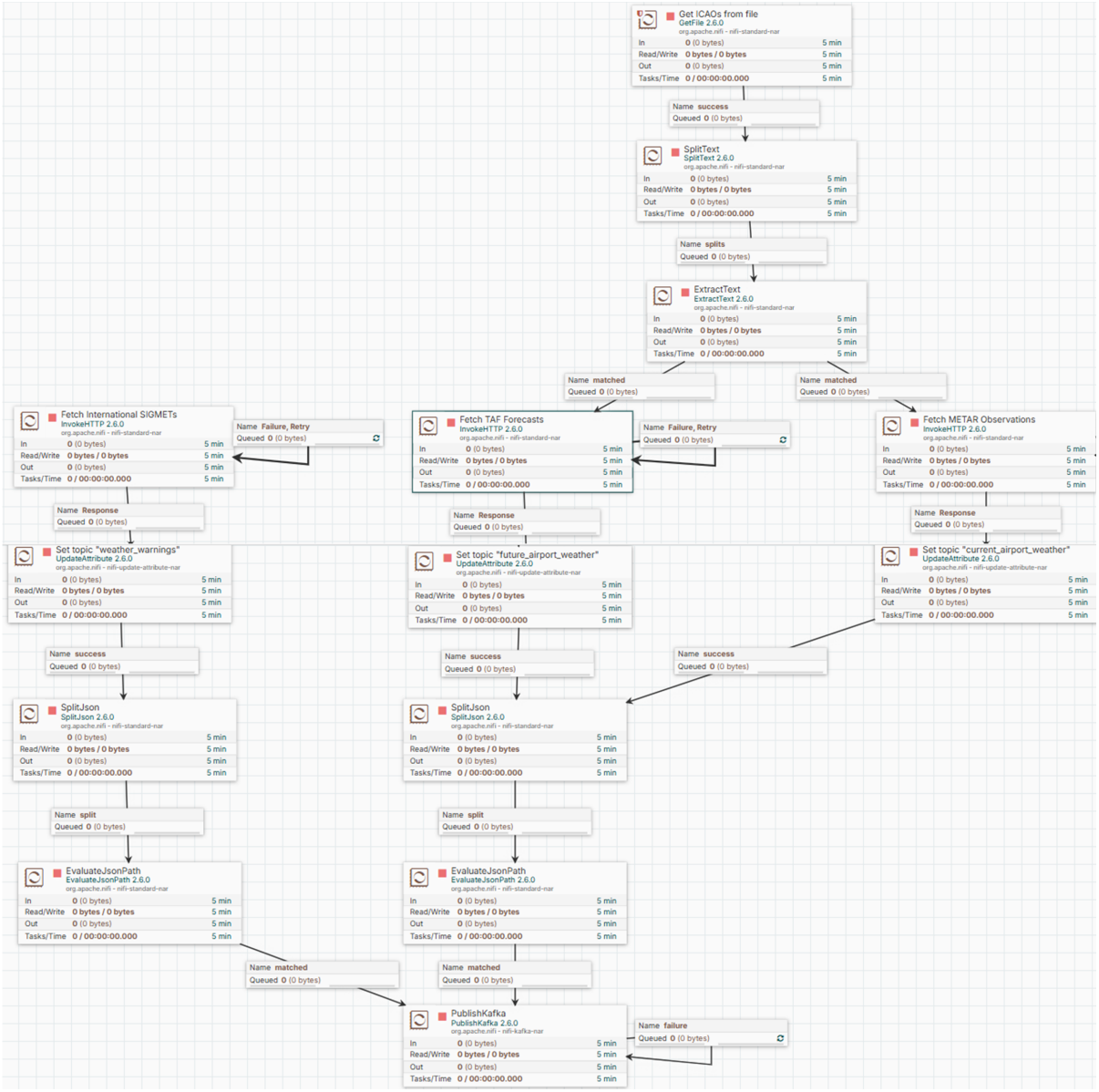

Figure 3: Aviation Weather flow in Nifi UI

METAR Stream (`metar` topic): A timer-driven `InvokeHTTP` processor polls the `/api/data/metar` endpoint every 5 minutes, retrieving current surface weather observations in GeoJSON format. The API returns a FeatureCollection where each Feature represents a METAR report with properties including station ID, observation time, temperature, dewpoint, wind direction/speed, visibility, ceiling, and cloud layers. The `SplitJson` processor extracts individual Features, with each METAR report maintaining its GeoJSON structure containing both meteorological properties and geographic coordinates. `EvaluateJsonPath` extracts the station ICAO identifier for use as the Kafka message key, enabling airport-based partitioning. `PublishKafka` transmits the complete GeoJSON Feature objects to preserve both weather data and spatial information for downstream correlation with nearby flight operations.

TAF Stream (`taf` topic): Operating on a 30-minute schedule, this flow ingests Terminal Aerodrome Forecasts from `/api/data/taf`. TAF responses contain multiple forecast periods for each airport, with each period represented as a separate Feature object. The `SplitJson` processor extracts these features, preserving the `timeGroup` field that identifies forecast period sequencing. Wind, visibility, ceiling, and cloud forecasts are retained in the properties object, while the geometry provides airport coordinates.

SIGMET Stream (`isigmet` topic): This flow polls `/api/data/isigmet` hourly for International SIGMETs warning of hazardous weather conditions (turbulence, icing, convection, volcanic ash). Each SIGMET Feature contains a Polygon or MultiPolygon geometry defining the affected airspace, with properties describing the hazard type, severity qualifier, altitude range, movement direction/speed, and validity period. The ICAO identifier of the issuing meteorological office serves as the message key. The complete GeoJSON structure enables spatial queries correlating SIGMETs with flight trajectories passing through affected regions.

3.1.2 Flow Deployment Automation

Manual flow construction in production environments introduces consistency risks and deployment delays. The project implements automated deployment via NiFi REST API orchestration and flow definition templates.

Flow templates define processor configurations, connections, and controller services in portable JSON format exported from the NiFi UI. The `deploy_flow.py` script automates deployment:

1. Authenticates to NiFi REST API using credentials from environment variables
2. Waits for NiFi availability via healthcheck polling of `/nifi` endpoint
3. Retrieves root process group identifier for flow instantiation
4. Uploads flow definition JSON files
5. Extracts controller service identifiers from the instantiated process group
6. Enables controller services
7. Starts all processors within the process group

The deployment container (`flow-deployer` service in Docker Compose) executes on NiFi startup, ensuring flows are provisioned automatically in fresh environments. This infrastructure-as-code approach enables version-controlled flow definitions, rapid environment replication, and disaster recovery through declarative configuration.

3.2 Message Streaming Layer (Kafka & ZooKeeper)

Apache Kafka serves as the distributed streaming platform, providing durable, scalable, and fault-tolerant message transport between ingestion and processing layers.

3.2.1 Kafka Cluster Configuration

The messaging backbone comprises a three-broker distributed cluster coordinated by a single ZooKeeper instance. This topology provides high availability and fault tolerance within a containerized environment, utilizing a dedicated Docker bridge network for efficient internal communication and external accessibility. Each broker operates in an isolated container with dedicated persistent volumes ensuring data remains intact across container restarts and enabling independent scaling of storage resources.

Listeners Architecture The cluster implements a sophisticated networking strategy with two distinct listener configurations to support both containerized microservices and host-based tools. The INTERNAL listener operates on the same port across all brokers, facilitating high-speed, low-latency data streaming between Docker-native clients such as NiFi producers and Spark consumers. Brokers advertise themselves within the container network using service discovery names, enabling seamless inter-service communication without port conflicts. Concurrently, the OUTSIDE listener maps unique external ports to the host machine, allowing external monitoring tools, CLI utilities, and management dashboards to interact with the cluster from the host environment. The `KAFKA_LISTENER_SECURITY_PROTOCOL_MAP` and `KAFKA_INTER_BROKER_LISTENER_NAME` configurations ensure that inter-broker replication exclusively uses the internal network, preventing unnecessary routing through the host layer and optimizing throughput.

Replication Strategy and Durability Guarantees To ensure data integrity and availability, the cluster enforces a comprehensive replication contract across multiple configuration layers. Topic data inherits a default replication factor of 3, guaranteeing that each partition maintains three synchronized replicas distributed across the broker ensemble. The durability contract is strengthened through `KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR=3` and `KAFKA_TRANSACTION_STATE_LOG_MIN_ISR=2`, which require transactional metadata to be replicated to three brokers with at least two in-sync replicas before commitment.

Storage Lifecycle and Topic Management The cluster maintains a 7-day time-based retention period coupled with topic-level compression (`compression.type=snappy`) to optimize storage efficiency while preserving operational data for near-term analytics. ZooKeeper operates with persistent volumes for data and transaction logs, managing critical cluster metadata including broker registrations, dynamic configurations, parti-

tion leader election, and consumer group coordination. The `ZOOKEEPER_TICK_TIME` parameter establishes the fundamental synchronization interval (2 seconds) for heartbeat mechanisms, session timeouts, and quorum-based consensus protocols ensuring consistent cluster state across distributed operations.

3.2.2 Kafka Topic and Client Configuration

Kafka Topic The messaging layer is structured around six Kafka topics, each representing a distinct data stream with tailored configurations (`flights_raw`, `flight_data`, `flight_track`, `current_airport_weather`, `future_airport_weather`, and `weather_warnings`). This design ensures that high-velocity flight telemetry and weather data are balanced across the three-broker cluster while maintaining strict data ordering and durability. All topics are configured with **six partitions**. In a three-broker cluster, this allows each broker to act as the leader for two partitions per topic, optimizing resource utilization.

Producer Data ingestion is handled through Apache NiFi using PublishKafka processors. The NiFi flows implement semantic partitioning using message keys to ensure related events are routed to the same partition.

Consumer Spark Structured Streaming consumes these topics using the Kafka connector. The Spark consumer configuration leverages Kafka's distributed consumer group mechanism while delegating offset management to Spark's checkpointing mechanism, ensuring fault-tolerant exactly-once processing semantics. Checkpoints are stored externally, allowing Spark to recover consumer state and resume processing from the last committed offset in case of failures.

3.3 Stream Processing Layer (Spark Structured Streaming)

Apache Spark Structured Streaming provides the computational engine for continuous data transformation, aggregation, and enrichment. The processing layer is designed to handle high-velocity streams while ensuring fault tolerance and exactly-once processing semantics through its declarative API.

3.3.1 Spark Application Design

The core of the processing layer is implemented in the `spark/reader.py` module. This application acts as a central transformation engine, following functional programming principles to maintain a clear separation between source consumption, transformation logic, and multi-sink orchestration.

Application Architecture and Orchestration The application is built using the **PySpark Structured Streaming API**, initialized via a `SparkSession` configured for

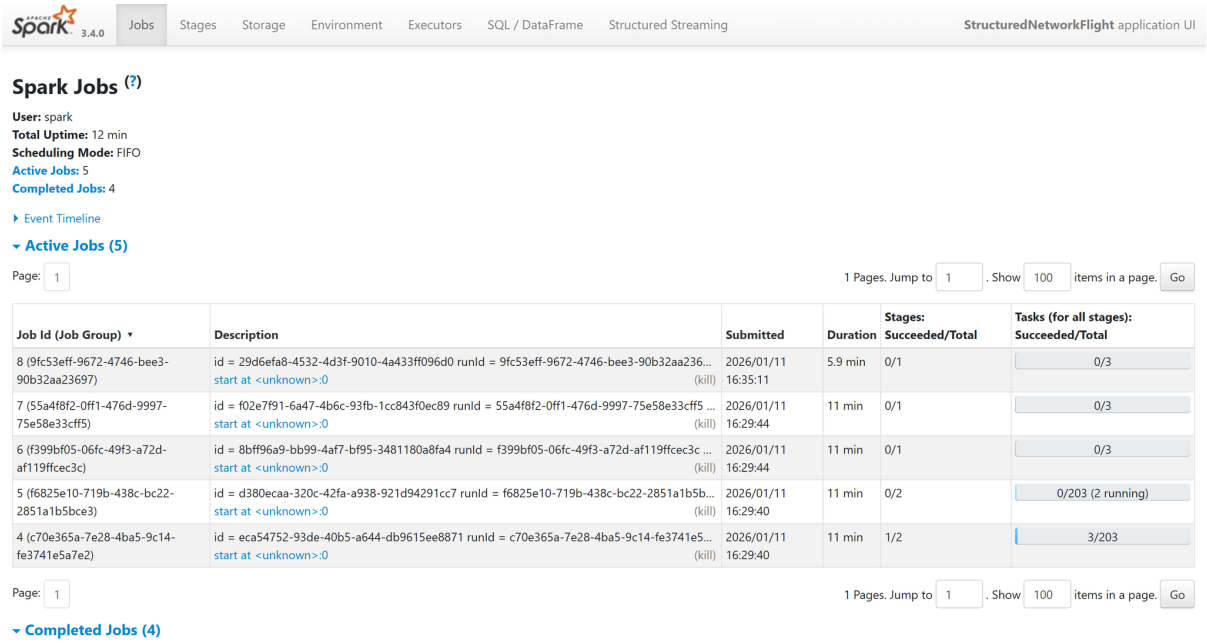

Figure 4: Spark Jobs Dashboard

high-throughput connectivity with a three-node Apache Cassandra cluster and MinIO (S3-compatible storage). To optimize development resources, the application runs in a containerized local mode with four worker threads, allowing for parallel processing of Kafka partitions. The orchestration logic maintains eight concurrent streaming queries (five writing to Cassandra and three to MinIO) and utilizes `spark.streams.awaitAnyTermination()` to manage the shared lifecycle of these asynchronous tasks.

Dependency and Infrastructure Configuration The system employs a hybrid dependency management strategy to bridge the JVM-based Spark core with the Python-based analytical ecosystem:

- **JVM Connectors:** Managed via the `-packages` flag, including `spark-sql-kafka`, `spark-cassandra-connector`, and `MinIO storage` for S3A filesystem support.
- **Python Libraries:** Integrated directly into the Docker image, featuring geographic processing tools (`airportsdata`, `reverse_geocode`) and performance-critical libraries (`pyarrow` and `pandas`) for vectorized operations.

Multi-Stream Ingestion Pattern The ingestion logic is encapsulated within three dedicated reader functions: `read_flight_stream`, `read_flightinfo_stream`, and `read_track_stream`. These functions implement strict **Schema Enforcement** using Spark's `StructType` specifications. By explicitly declaring field names, types, and nullability, the system prevents silent schema evolution issues and ensures that the downstream transformation functions receive consistent data structures.

Transformation Pipeline The application defines modular transformation functions explicitly mapped to target Cassandra tables. The `get_aircrafts_by_icao` function performs column projection for current state snapshots, while `get_aircraftstates_by_icao24` leverages the `explode()` function to denormalize nested flight trajectory arrays into individual waypoint records for time-series analysis. For statistical views, functions such as `get_activeaircraft_by_country_hour` and `get_departures_by_country_hour` prepare data for aggregation by extracting temporal buckets and applying geographical enrichment logic.

3.3.2 Data Cleaning and Enrichment

Raw telemetry data undergoes a multi-stage validation and enrichment process. While preliminary filtering of incomplete records (e.g., null airport codes) is handled upstream at the NiFi ingestion layer, the Spark layer focuses on robust input validation within its enrichment logic. Two specialized **Vectorized Pandas UDFs** (User Defined Functions) handle geographical resolution:

- `country_code_from_icao_udf`: Resolves airport identifiers using the `airportsdata` library to support terminal-based analytics.
- `country_code_from_latlon_udf`: Executes **offline reverse geocoding** via the `reverse_geocode` library. This function implements defensive error handling, explicitly catching `NaN`, infinity, or null coordinates and returning "Unknown." This strategy ensures stream continuity and maintains a resilient pipeline even when sensor data is noisy or incomplete.

These UDFs allow for efficient batch processing of coordinate arrays across executors rather than row-by-row invocation, significantly reducing the overhead of Python-to-JVM communication.

3.3.3 Windowing, Watermarking, and Aggregations

Temporal analytics within the Spark processing layer require windowing operations to group unbounded telemetry streams into finite intervals. The system implements a dual-windowing strategy using non-overlapping **Tumbling Windows**, aligned with Unix epoch boundaries to support both operational and analytical use cases.

Windowing and Watermarking Strategy The implementation distinguishes between immediate situational awareness and long-term trend analysis through tailored window sizes and watermarking thresholds:

- **1-Minute Tumbling Windows:** Applied to the `aircrafts_by_cell_minute` stream to support the Live Map. This windowing logic uses a strict **10-second watermark** to minimize display latency, ensuring that position updates are reflected on

the dashboard as quickly as possible, even at the cost of discarding severely delayed packets.

- **1-Hour Tumbling Windows:** Applied to statistical streams such as `activeaircraft`, `departures`, and `arrivals`. These windows utilize a **10-minute watermark** to accommodate significant network delays or late-arriving batch data, prioritizing data completeness for historical reporting.

Aggregation Logic The application leverages specific Spark SQL functions to optimize performance and handle the inherent imperfections of sensor-derived data:

- **Cardinality Estimation:** The system uses the `approx_count_distinct()` function, based on the **HyperLogLog** algorithm, to estimate the count of unique aircraft per country. This provides a high-performance cardinality estimate with an acceptable 1-2% error bound, significantly reducing the memory footprint compared to exact `countDistinct` operations.
- **State Consolidation:** For position and kinematics data, the system employs `last(col, ignorenulls=True)` within the 1-minute window to retain the most recent valid sensor reading.
- **Temporal Bucketing:** Aggregated results are indexed by `country_code` and `time_bucket` (e.g., `minute_bucket`, `hour_bucket`). This transforms the dynamic window into a static integer key in order to write for queries in Apache Cassandra.

Output Semantics and Integrity To ensure data consistency, all windowed queries operate in **Append mode**. In this configuration, results are only committed to the storage layer after the watermark has passed and the window is officially "closed." This ensures that once a statistic is written to Cassandra or MinIO, it is complete and final, preventing the complexities of updating historical records in a distributed environment.

3.4 Storage Layer (Dual Storage Strategy)

The system implements a dual-path storage architecture separating hot operational data from cold archival data, optimizing query performance and cost efficiency.

3.4.1 Cassandra (Operational / Hot path)

Apache Cassandra serves as the primary operational database for the system's *Hot Path*, providing high-throughput write capabilities and low-latency access to processed aggregates and real-time aircraft states.

Cluster Topology and Replication Strategy The deployment consists of a three-node distributed cluster, ensuring high availability and resilience. To guarantee that the system can withstand a node failure without data loss, the `flight_analytics` keyspace is configured with **NetworkTopologyStrategy** and a **Replication Factor (RF) of 3**. This means every piece of data is mirrored across all three nodes in the cluster. Each node operates within a dedicated Docker container utilizing persistent volumes for commit logs and data directories, ensuring durability across container lifecycles.

Query-Driven Schema Design Unlike traditional relational databases, the Cassandra schema is designed using a **query-first approach**, where data is denormalized into specific tables to satisfy predefined query patterns without the need for costly JOIN operations.

Below is the Chebotko diagram for our Cassandra model schema, describing the query-driven schema architecture, detailing the relationship between our access patterns and the physical storage structures to ensure efficient data retrieval and minimal partition scanning.

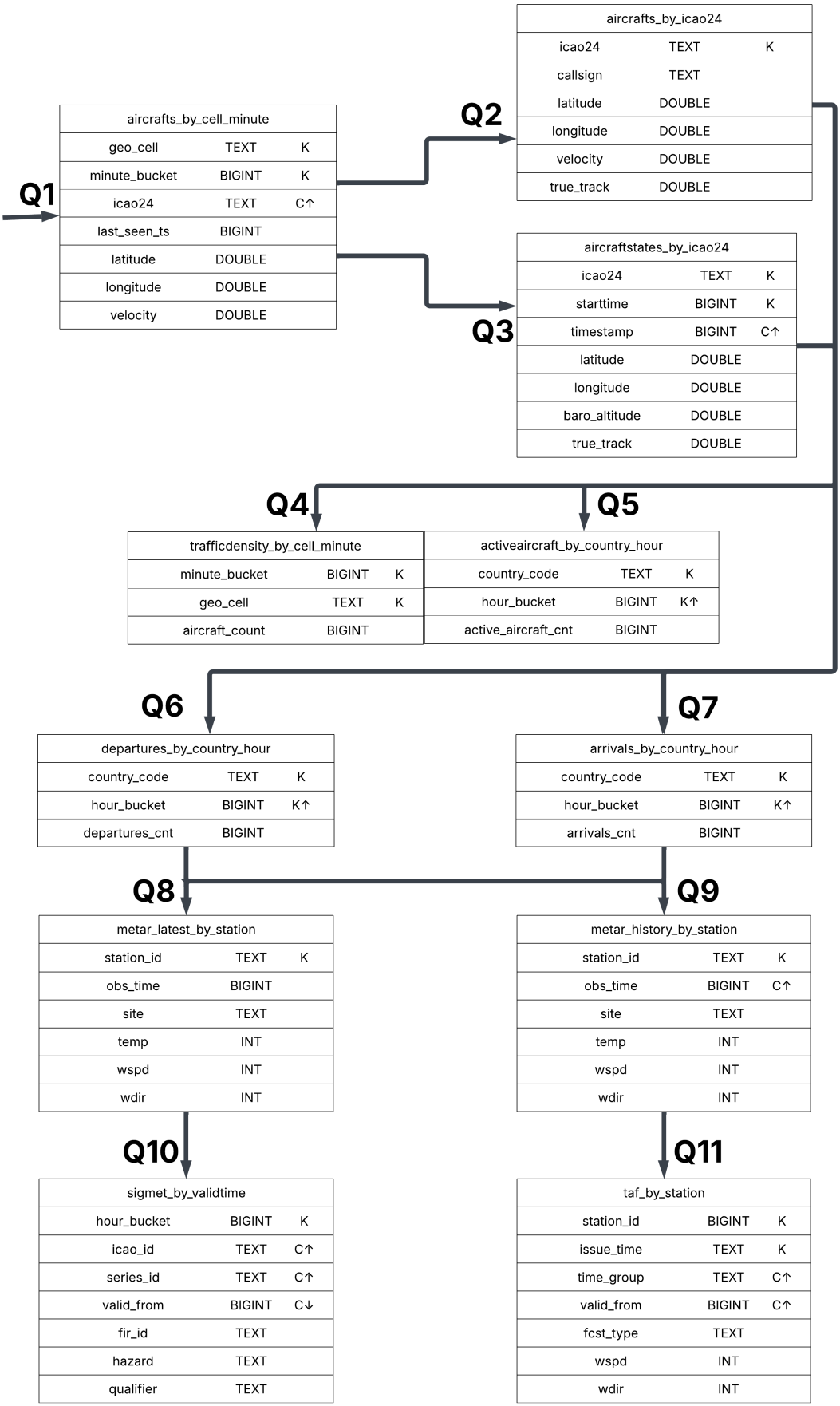

Figure 5: Chebotko Diagram

1. Real-time Flight Tracking Data Flow This group of tables serves interactive queries on maps and flight details, designed according to a "drill-down" flow (querying progressively deeper details):

- **Q1 (Spatial Query):** Query the `aircrafts_by_cell_minute` table.
  - *Purpose:* Retrieve a list of active aircraft within a specific spatial region (`geo_cell`) and time frame (`minute_bucket`).
  - *Keys:* Uses a dual partition key (`geo_cell`, `minute_bucket`) to distribute spatial data over time, avoiding the "hot partition" phenomenon.
- **Q2 (Detail Lookup):** Query the `aircrafts_by_icao24` table.
  - *Purpose:* Using the list of `icao24` IDs obtained in Q1, retrieve detailed current status information (position, velocity, callsign) for each specific aircraft.
  - *Keys:* `icao24` acts as the unique partition key, facilitating point lookups with low latency $O(1)$.
- **Q3 (Trajectory History):** Query the `aircraftstates_by_icao24` table.
  - *Purpose:* Retrieve the flight history (trajectory) of a specific flight.
  - *Keys:* The Partition Key is a composite of (`icao24`, `starttime`) to isolate data for each flight. The Clustering Key is the `timestamp` (or time), sorted in ascending order ($C \uparrow$), to support reconstructing the flight path chronologically.

2. Aggregated Analytics Flow This group of tables stores pre-aggregated data calculated by Spark based on time windows, serving Dashboard visualizations:

- **Q4 (Traffic Density):** The `trafficdensity_by_cell_minute` table supports displaying traffic density (`aircraft_count`) by geographical grid and minute.
- **Q5, Q6, Q7 (Country Statistics):** Tables: `activeaircraft...`, `departures...`, and `arrivals_by_country_hour`.
  - *Purpose:* Statistics on the number of active aircraft, departures, and arrivals by country.
  - *Design:* Uses `country_code` as the Partition Key (K) to group data by country, and `hour_bucket` as the Clustering Key ($K \uparrow$) to query time-series data for trend charting.

3. Aviation Weather Integration Data Flow The system extends storage to include aviation meteorological data to serve correlation analysis:

- **Q8 & Q9 (METAR):** The `metar_latest...` table stores current observations ($K =$ `station_id`), and `metar_history...` stores weather history ($C \uparrow =$ `obs_time`) at each airport station.
- **Q10 (SIGMET):** The `sigmet_by_validtime` table stores hazardous weather warnings, partitioned by time frame (`hour_bucket`) and sorted by validity time (`valid_from`).
- **Q11 (TAF):** The `taf_by_station` table stores weather forecasts at airports, supporting queries for forecast bulletins by issuance time.

Legend:

- **K (Partition Key):** Determines the node where data is stored.
- **C / K↑ (Clustering Key):** Determines the sort order of data on the disk (the upward arrow indicates ascending order).
- **Arrow ($Q1 \rightarrow Q2$):** Represents the business flow; the result from the previous query provides input parameters for the subsequent query.

Consistency Model and Write Path In accordance with the **CAP Theorem**, the system prioritizes **Availability** and **Partition Tolerance (AP)** to meet the requirements of real-time monitoring.

- **Consistency Level:** The system is configured with a consistency level of **ONE** for both read and write operations. This minimizes latency and ensures that the system remains responsive even under heavy ingestion loads.
- **Spark Integration:** The `spark-cassandra-connector` writes streaming results using the `append` output mode.
- **Idempotent Upserts:** Reliability is maintained through Cassandra's native **UPSERT semantics**. Since writes are based on the Primary Key, any re-processed data from Spark (due to a failure or retry) will simply overwrite the existing record, ensuring **exactly-once processing semantics** at the storage layer without duplicate entries.

3.4.2 MinIO (Archival / Cold Path)

MinIO object storage serves as the fundamental Cold Store or Batch Layer within the system's Lambda Architecture. It provides an S3-compatible environment to persist raw and processed data for long-term historical analysis, disaster recovery, and data replay (backfilling).

Bucket Organization and System State The system's data lake is structured into seven distinct buckets, each serving a specific role in the data lifecycle. The `flight-raw` bucket stores the original, high-volume state vectors from OpenSky Network API, while `flight-data` contains flight segment metadata and terminal information such as departure and arrival events. Detailed flight trajectories and waypoints are preserved in the `flight-tracks` bucket. Three additional buckets support weather data archival: `weather-metar` archives historical METAR (Meteorological Aerodrome Report) observations for retrospective weather analysis, `weather-taf` stores TAF (Terminal Aerodrome Forecast) predictions for forecast accuracy validation, and `weather-isigmet` preserves SIGMET (Significant Meteorological Information) and AIRMET hazard warnings with associated geospatial polygons. The `checkpoints` bucket serves a critical role in system reliability by storing Spark Structured Streaming metadata, including Kafka offsets and window states for stateful operations.

Storage Format and Partitioning Strategy To optimize analytical performance and storage efficiency, the system converts raw JSON payloads into Apache Parquet, a column-oriented storage format. The application utilizes the Snappy compression algorithm, providing an optimal balance between high-speed read/write performance and significant disk space reduction. Data is organized hierarchically using a time-based Hive-style partitioning scheme following the pattern `year=YYYY/month=MM/day=DD`.

Write Mechanism and Trigger Intervals The Spark processing layer communicates with MinIO using the S3A FileSystem protocol (`fs.s3a.impl`). Spark is configured with distinct trigger intervals based on data volume and update frequency. High-velocity streams (`flights_raw`, `flight_tracks`, and all weather streams) utilize a 5-minute interval to ensure data is archived frequently without overwhelming the storage layer. In contrast, the lower-volume `flight-data` metadata stream operates on a 10-minute interval, allowing for larger, more efficient file accumulation and reduced object storage overhead.

Fault Tolerance and Operational Context By storing metadata and state information in the `checkpoints` bucket, the system achieves fault tolerance and ensures exactly-once processing semantics. In the event of a crash, Spark can resume processing exactly where it left off by reading the stored offsets from MinIO. In the current development environment, the system operates with default root credentials (`minioadmin`) and manual data cleanup policies.

3.5 Analytics & Visualization Layer

The analytics layer bridges storage and presentation through federated querying and interactive visualization.

3.5.1 Trino Query Engine

Trino (formerly PrestoSQL) functions as a distributed SQL query engine that serves as a high-performance **SQL Abstraction Layer**. It provides an ANSI SQL interface over Apache Cassandra, enabling complex analytical queries that would otherwise be restricted by the limitations of the Cassandra Query Language (CQL).

Architecture and Kubernetes Deployment The Trino cluster utilizes a Kubernetes-native architecture consisting of a Coordinator node and a Worker node. The Coordinator is responsible for parsing incoming SQL statements, optimizing execution plans, and managing the worker pool. The Worker nodes execute query fragments and fetch data from the storage connectors in parallel. To ensure stability within the shared cluster, the JVM heap is capped at 2GB, balancing memory-intensive shuffles with the available infrastructure resources.

Cassandra Connector and Service Discovery The system defines a dedicated catalog via the `cassandra.properties` configuration. Unlike local Docker setups, this deployment leverages **Kubernetes DNS** for dynamic service discovery, allowing Trino to interact with the Cassandra cluster through a unified service endpoint:

- `connector.name=cassandra`
- `cassandra.contact-points=cassandra.default.svc.cluster.local`
- `cassandra.load-policy.dc-aware.local-dc=dc1`
- `cassandra.consistency-level=ONE`

By setting the consistency level to **ONE** and utilizing a **DC-aware load balancing policy** (targeting the `dc1` data center), the system minimizes network latency and prioritizes high read availability—aligning with the system's requirements for responsive real-time dashboards.

Query Optimization and Predicate Pushdown Trino's efficiency is primarily driven by its ability to perform **Predicate Pushdown**. This optimization ensures that SQL filters are translated into CQL `WHERE` clauses and executed directly at the Cassandra storage layer, rather than fetching full tables into memory.

For example, when querying hourly statistics, the engine leverages the Clustering Keys defined in the Cassandra schema:

```
SELECT country_code, SUM(active_aircraft_cnt)
FROM activeaircraft_by_country_hour
```

```
WHERE hour_bucket >= 1672531200
GROUP BY country_code
```

In this scenario, Trino pushes the `hour_bucket` predicate down to the storage nodes. Because `hour_bucket` is a part of the primary key structure, Cassandra can efficiently locate the specific time partitions, drastically reducing the amount of data transferred across the network.

Network Interface and Integration The Trino service is exposed via a **NodePort (Port 8080)**, allowing **Apache Superset** to connect as a downstream client. This connection is established through the `SQLAlchemy` driver, providing a seamless bridge between the raw distributed storage and the interactive visualization layer.

3.5.2 Apache Superset Visualization

Apache Superset serves as the primary visualization and Business Intelligence (BI) layer, providing an interactive interface for real-time monitoring and historical data exploration. By decoupling the presentation layer from the storage engines, it allows end-users to derive insights through a rich library of charts and geospatial visualizations.

Deployment Architecture The Superset stack operates as a multi-tier containerized application on Kubernetes, comprising three primary runtime components:

- **Superset Web Server:** A Flask-based application that handles the React-based frontend, user authentication, and SQL query execution.
- **Metadata Store (PostgreSQL):** A persistent database (deployed via StatefulSet) used to store dashboard definitions, chart configurations, user roles, and dataset metadata.
- **Caching Layer (Redis):** An in-memory store used to cache query results. This significantly reduces the load on the Trino-Cassandra pipeline and ensures that frequently accessed dashboards render with minimal latency.

Data Source Integration and Initialization Data integration is achieved through a **Trino-exclusive connection** using the `sqlalchemy-trino` driver. The system follows a "zero-click" deployment strategy where a transient **Init Job** (`superset-init`) automatically executes during startup to migrate database metadata, create admin credentials, and register datasets via the Trino service URL (`trino.trino.svc.cluster.local`).

Semantic Layer and Dataset Mapping The platform maps the underlying Cassandra tables into six distinct Superset Datasets. This semantic layer allows the system to pre-define dimensions and metrics, ensuring that the complex NoSQL structure is presented in a user-friendly format:

- **Real-time Monitoring:** Datasets such as `aircrafts_by_cell_minute` and `activeaircraft_by_country_minute` support live situational awareness.
- **Historical Trends:** `activeaircraft_by_country_hour` and `departureaircraft_by_icao24_day` enable long-term operational analysis and reporting.

Visualization Figure 6 illustrates the main Superset dashboard named "Live Airspace Monitor" built for the Flight Data Monitoring system. The dashboard is designed as an executive overview for air-traffic activity and is composed of four primary panels:

- **Total flights KPI (top-left):** A Big Number visualization summarizes the total number of flights observed in 2026. This metric provides an immediate snapshot of global traffic volume.
- **Flights by country map (top-right):** A choropleth world map visualizes the geographic distribution of flight activity by country. Countries are shaded by flight count (darker indicates higher activity), enabling quick identification of high-traffic regions and comparative traffic intensity across continents.
- **Active aircraft KPI (bottom-left):** A second Big Number visualization presents the number of active aircraft in 2026. This metric is derived from hourly/minute aggregation logic in the processing layer.
- **Flights per hour time series (bottom-right):** A line chart shows the evolution of traffic intensity over time. In our case, the dashboard plots an hourly aggregation against the hour bucket, enabling operators to observe surges, troughs, and temporal patterns in aircraft activity.

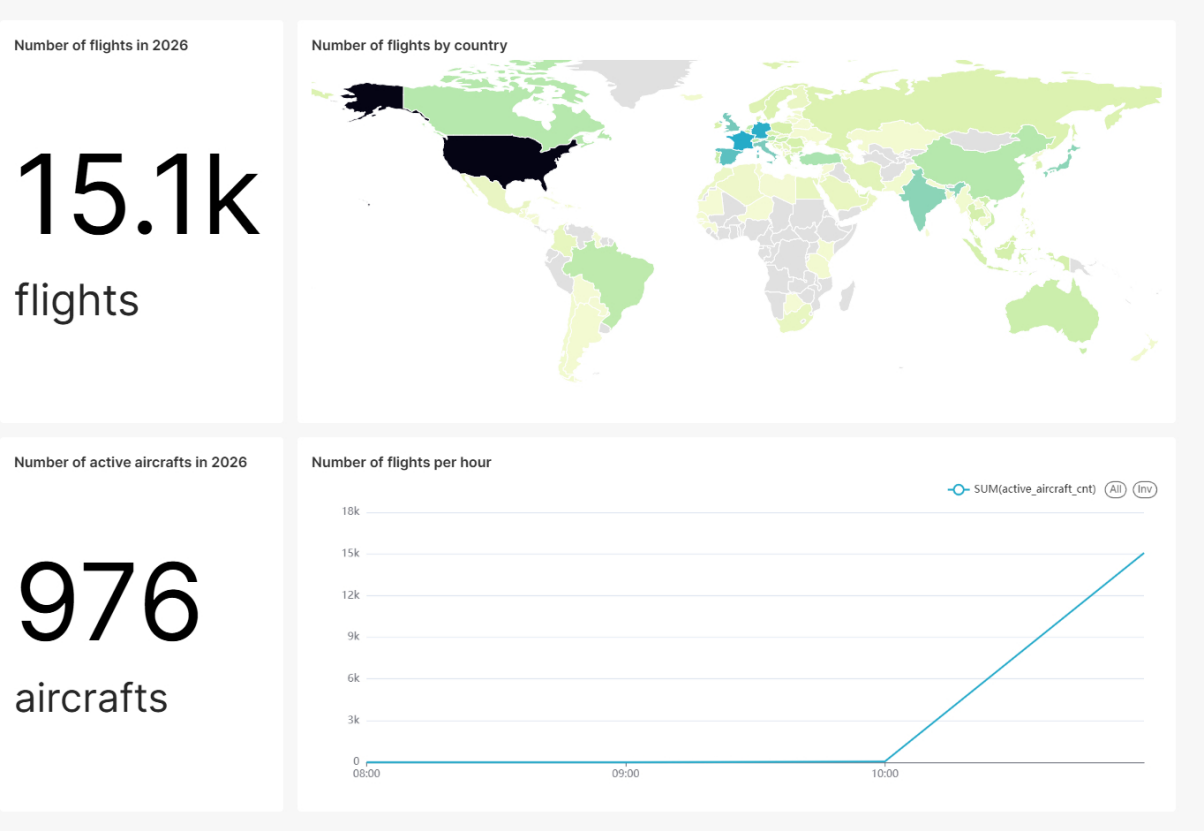

Figure 6: Superset dashboard overview showing total flights, flights by country (choropleth map), active aircraft, and hourly flight activity.

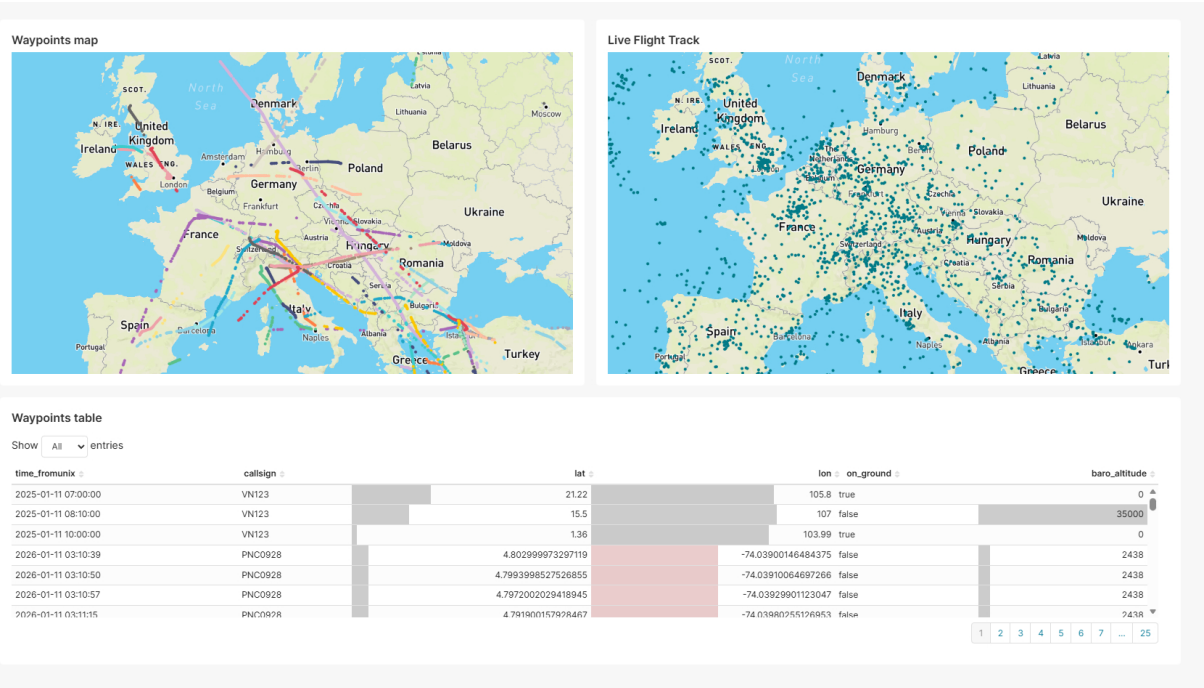

Figure 7: Superset dashboard displaying aircraft waypoint trajectories, real-time flight positions across Europe, and a detailed waypoints table with timestamps, callsigns, coordinates, ground status, and altitude.

Performance and Caching Strategy To maintain a responsive user experience, the system implements multi-level caching via Redis. Query results are stored with configurable TTL (Time-To-Live) strategies. For real-time monitoring dashboards, shorter TTLs are used to ensure data freshness, while historical reports utilize longer durations to optimize performance. This architecture effectively decouples the dashboard rendering speed from the underlying read performance of the distributed Cassandra cluster.

3.6 Deployment & Operations Approach

Local deployment (Docker Compose vs. Minikube). Docker Compose provides the fastest local loop: services are co-located on a single host, networking is implicit through a shared Docker network, and configuration is handled via the `.env` file and compose files. This is ideal for quick smoke tests but does not exercise Kubernetes primitives (namespaces, PVCs, service discovery, or Helm releases). Minikube is closer to production because it runs the same Kubernetes manifests, ConfigMaps/Secrets, and resource constraints used in Google Kubernetes Engine (GKE). It validates scheduling, service discovery, and persistent volumes locally while remaining a single-node setup. To run this project locally, a powerful host computer is recommended (64GB RAM and at least 200GB of disk).

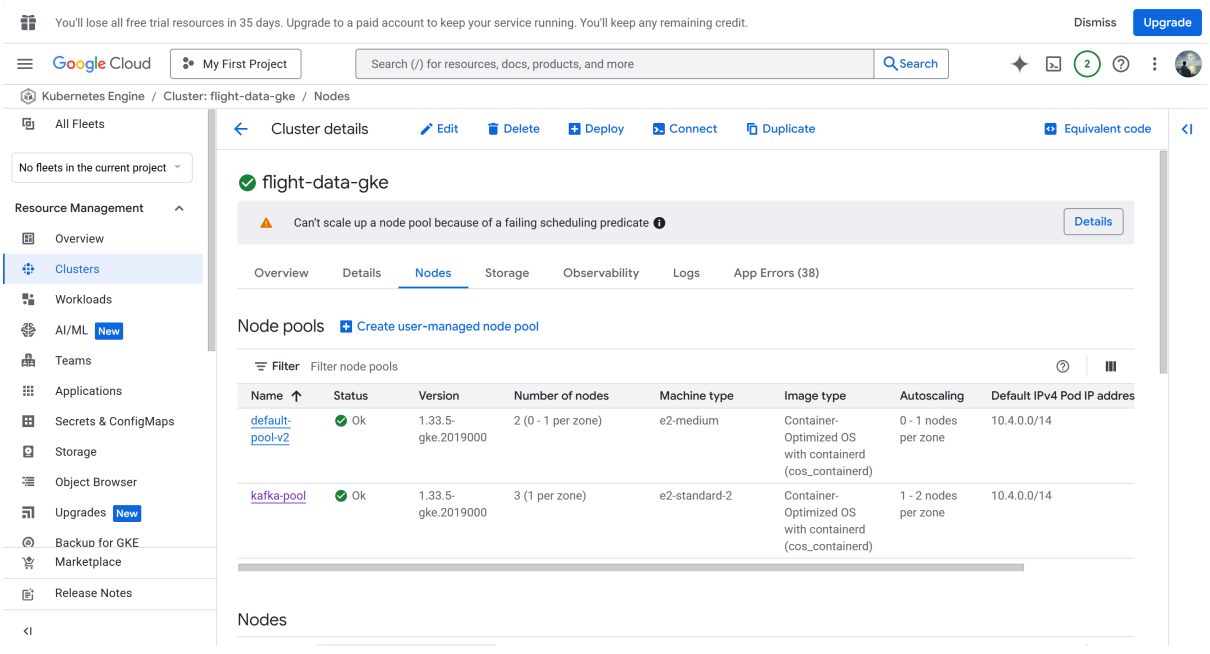

Figure 8: The cluster management tab in Google Cloud Console. We use two node pools: `kafka-pool` with e2-standard-2 instances (2 vCPU, 8GB RAM) for stateful workloads, and `default-pool-v2` with e2-medium instances (2 vCPU shared, 4GB RAM) for stateless services.

Production deployment using Google Kubernetes Engine (GKE). GKE is the production-grade target with multi-namespace isolation (seven dedicated namespaces:

`kafka`, `cassandra`, `spark`, `trino`, `nifi`, `minio`, and `superset`), persistent storage classes (`pd-ssd` for stateful workloads, `pd-balanced`/`standard-rwo` for others), and a managed control plane. Custom Kubernetes manifests handle all deployments, with configuration centralized in ConfigMaps and Secrets so the same container images work across environments.

Node Pool Architecture. The cluster uses two node pools with autoscaling enabled:

- **kafka-pool**: Three e2-standard-2 instances (2 vCPU, 8GB RAM, 50GB SSD each) hosting stateful workloads requiring high memory and I/O—Kafka brokers, Cassandra, ZooKeeper, and Spark streaming.
- **default-pool-v2**: e2-medium instances (2 shared vCPU, 4GB RAM, 50GB standard disk) with autoscaling from 0–3 nodes, hosting stateless services like Trino workers and auxiliary pods.

Component Deployment Details. Table 2 summarizes the Kubernetes resource types, replica counts, and resource allocations for each component.

Table 2: GKE Deployment Configuration per Component

Component	K8s Type	Replicas	CPU (req/lim)	Memory (req/lim)	Storage
Kafka	StatefulSet	3	500m / 1	1Gi / 2Gi	10Gi pd-ssd
ZooKeeper	StatefulSet	1	100m / 500m	256Mi / 512Mi	–
Cassandra	StatefulSet	1	300m / 1	1Gi / 2Gi	10Gi pd-ssd
Spark Streaming	Deployment	1	500m / 2	2Gi / 4Gi	–
NiFi	Deployment	1	500m / 2	2Gi / 3Gi	20Gi pd-balanced
MinIO	Deployment	1	100m / 500m	256Mi / 512Mi	20Gi standard-rwo
Trino Coordinator	Deployment	1	200m / 1	1Gi / 1.5Gi	–
Trino Workers	Deployment	3	300m / 1	1.5Gi / 2Gi	–
Superset	Deployment	1	200m / 500m	768Mi / 1.5Gi	–
Superset Postgres	StatefulSet	1	50m / 200m	128Mi / 256Mi	1Gi pd-balanced
Superset Redis	Deployment	1	–	–	–

Service Exposure.

- **LoadBalancer**: Superset is exposed externally via a GCP LoadBalancer on port 80, providing public access for dashboard users.
- **NodePort**: NiFi (port 8443) and Trino (port 8080) use NodePort services for administrative access via `kubectl port-forward` or direct node IP.
- **ClusterIP**: All other services (Kafka, Cassandra, ZooKeeper, MinIO, internal Trino workers) use ClusterIP for internal cluster communication only.
- **Headless Services**: Kafka and Cassandra use headless services (`*-headless`) for StatefulSet pod DNS resolution, enabling direct pod-to-pod communication required by their clustering protocols.

Stateful Workload Considerations. Kafka and Cassandra are deployed as StatefulSets with stable network identities and persistent volume claims:

- **Kafka**: Three brokers with 10Gi pd-ssd volumes each. Default topic replication factor is configured for the 3-broker cluster.
- **Cassandra**: Single-node deployment with `SimpleStrategy` replication factor of 1 (suitable for development; production would use `NetworkTopologyStrategy` with RF=3).
- **ZooKeeper**: Single-node for Kafka coordination (production would use 3-node quorum).

Why GKE over Minikube. Running the pipeline on GKE provides multi-node elasticity, managed control plane reliability, and production-grade storage and networking that Minikube cannot offer. GKE supports horizontal pod autoscaling, node pool separation for stateful vs. stateless workloads, and persistent volumes backed by SSD storage classes, enabling safe upgrades and predictable performance under load. This is critical when the pipeline must withstand real traffic spikes or sustained high ingest rates, which would otherwise exhaust a single-node Minikube setup.

Scalability under higher request rates and data velocity.

- *NiFi* scales by adding replicas to its Deployment and increasing concurrent processors (currently 1 replica with 2 CPU cores and 3GB memory limit).
- *Kafka* scales by adding brokers to the StatefulSet (currently 3 replicas) and increasing topic partitions for parallel consumption.
- *Spark Structured Streaming* scales by increasing executor count via the Deployment replica count and adjusting driver/executor memory (currently 4GB limit with 2 CPU cores).
- *Cassandra* scales by adding nodes to the StatefulSet and adjusting replication factor; write throughput scales linearly with node count.
- *MinIO* scales by adding replicas or migrating to GCS for managed elastic storage.
- *Trino* scales by adding worker replicas (currently 3 workers with 2GB memory each) to increase concurrent query capacity.
- *Superset* scales horizontally by running multiple web worker replicas and scaling its PostgreSQL metadata store and Redis cache.

On GKE, these scale-out actions are supported by cluster autoscaling, dedicated node pools with appropriate machine types, and persistent storage classes. Minikube is constrained by single-node resources and cannot scale beyond the host's limits.

4 Conclusion

4.1 Summary of Achievements and Discussion

This research project successfully engineered a robust, end-to-end real-time flight telemetry analytics system, addressing the challenges of high-velocity data processing through a modern distributed stack. The implementation delivers a production-viable architecture that provides unified stream processing, effectively eliminating the duplication of logic between batch and real-time layers.

The achievement of end-to-end **exactly-once semantics**—enabled by Kafka transactions and Spark checkpointing—ensures analytical precision despite potential infrastructure failures. Furthermore, the dual-storage implementation optimizes query performance via Apache Cassandra while maintaining long-term cost efficiency through MinIO archival. The system consistently achieves sub-minute latency from data generation to visualization, enabling operational dashboards that handle late-arriving data through robust watermarking strategies.

The project serves as a practical validation of the **five V's of Big Data**:

- **Volume:** Managing millions of daily events with terabyte-scale annual archival.
- **Velocity:** Sustaining high ingestion rates through a distributed messaging and processing backbone.
- **Variety:** Handling semi-structured JSON payloads with nested fields and schema evolution.
- **Veracity:** Mitigating GPS drift and transmission errors through vectorized validation UDFs.
- **Value:** Extracting actionable insights for air traffic density and regional movement patterns.

Challenges and Limitations The implementation highlighted several critical constraints:

- **Computational Enrichment Overhead:** Although offline libraries were utilized to eliminate API latency, the serialization overhead of Python-based UDFs within the Spark JVM remains a significant CPU bottleneck.
- **State Management Complexity:** Stateful operations require meticulous checkpoint management and TTL tuning to prevent memory exhaustion during sustained peak loads.

- **Integration Cognitive Load:** Orchestrating six major distributed technologies (NiFi, Kafka, Spark, Cassandra, Trino, and Superset) requires broad expertise and complex dependency management.

4.2 Future Work

Several enhancement opportunities have been identified to extend the system's capabilities toward an enterprise-grade production environment:

Advanced Analytics and Intelligence

- **Predictive Modeling:** Integrating **Spark MLlib** to develop models predicting flight delays based on historical congestion patterns and real-time weather data.
- **Streaming Anomaly Detection:** Implementing unsupervised models to identify unusual flight trajectories or transponder irregularities that may indicate safety or security concerns.

Performance and Operational Optimization

- **Distributed Caching Layer:** Introducing **Redis** to cache frequently accessed reference data (e.g., airport identifiers), significantly reducing UDF execution time.
- **Automated Scaling:** Implementing Kubernetes **Horizontal Pod Autoscalers (HPA)** to dynamically adjust processing capacity based on real-time Kafka consumer lag metrics.
- **Tiered Storage Management:** Implementing automated lifecycle policies to transition "cold" data to cheaper object storage tiers, optimizing long-term infrastructure costs.

Security Hardening and Governance

- **Zero Trust Architecture:** Implementing mutual TLS (mTLS) for all internal service-to-service communication and migrating to a centralized vault for secrets management.
- **Data Lineage tracking:** Utilizing tools like Apache Atlas to provide full visibility into data transformations from ingestion to visualization for compliance purposes.

User Experience Improvements

- **Real-time Alerting Framework:** Developing a push-based notification system via Webhooks or SMS responding to critical threshold breaches (e.g., emergency squawk codes).

- **High-Fidelity Geospatial UI:** Enhancing Superset dashboards with **3D altitude rendering** and sequential trajectory playback using the Deck.gl integration.

In conclusion, this project demonstrates that open-source Big Data technologies can be orchestrated to build sophisticated, real-time analytics platforms capable of handling global-scale telemetry. The architecture and methodologies documented herein provide a scalable blueprint for similar streaming systems across diverse industrial and IoT domains.

References

[1] M. Schäfer, M. Strohmeier, V. Lenders, I. Martinovic, and M. Wilhelm, "Bringing up OpenSky: A large-scale ADS-B sensor network for research," in *Proceedings of the 13th IEEE/ACM International Symposium on Information Processing in Sensor Networks (IPSN)*, 2014, pp. 83–94.

[2] J. Kreps, "Questioning the Lambda Architecture," O'Reilly Media, 2014. [Online]. Available: https://www.oreilly.com/radar/questioning-the-lambda-architecture/

[3] Apache Software Foundation, "Apache Kafka Documentation," 2024. [Online]. Available: https://kafka.apache.org/documentation/

[4] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[5] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *ACM SIGOPS Operating Systems Review*, vol. 44, no. 2, pp. 35–40, 2010.

[6] Apache Software Foundation, "Apache NiFi Documentation," 2024. [Online]. Available: https://nifi.apache.org/docs.html

[7] Trino Software Foundation, "Trino: Fast distributed SQL query engine," 2024. [Online]. Available: https://trino.io/docs/current/

[8] Apache Software Foundation, "Apache Superset Documentation," 2024. [Online]. Available: https://superset.apache.org/docs/intro

[9] MinIO, Inc., "MinIO High Performance Object Storage," 2024. [Online]. Available: https://min.io/docs/minio/linux/index.html

[10] The Linux Foundation, "Kubernetes Documentation," 2024. [Online]. Available: https://kubernetes.io/docs/

[11] D. Laney, "3D data management: Controlling data volume, velocity and variety," META Group Research Note, 2001.

[12] T. Akidau et al., "The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing," *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1792–1803, 2015.